

Base de données

En **informatique** , une **base de données** est un ensemble organisé de **données**, ou un type de **stockage de données**, reposant sur l'utilisation d'un **système de gestion de base de données** (**SGBD**). Ce **logiciel** interagit avec **les utilisateurs finaux** , **les applications** et la base de données elle-même pour capturer et analyser les données. Le SGBD comprend également les fonctionnalités essentielles permettant d'administrer la base de données. L'ensemble composé de la base de données, du SGBD et des applications associées constitue un **système de base de données** . Le terme « base de données » est souvent employé de manière plus générale pour désigner le SGBD, le système de base de données ou une application associée à la base de données.

```
dvdrntal=# select title, release_year, length, replacement_cost from film
dvdrntal=#   where length > 120 and replacement_cost > 29.50
dvdrntal=#   order by title desc;
```

title	release_year	length	replacement_cost
West Lion	2006	159	29.99
Virgin Daisy	2006	179	29.99
Uncut Suicides	2006	172	29.99
Tracy Cider	2006	142	29.99
Song Hedwig	2006	165	29.99
Slacker Liaisons	2006	179	29.99
Sassy Packer	2006	154	29.99
River Outlaw	2006	149	29.99
Right Cranes	2006	153	29.99
Quest Mussolini	2006	177	29.99
Poseidon Forever	2006	159	29.99
Loathing Legally	2006	140	29.99
Lawless Vision	2006	181	29.99
Jingle Sagebrush	2006	124	29.99
Jericho Mulan	2006	171	29.99
Japanese Run	2006	135	29.99
Gilmore Boiled	2006	163	29.99
Floats Garden	2006	145	29.99
Fantasia Park	2006	131	29.99
Extraordinary Conquerer	2006	122	29.99
Everyone Craft	2006	163	29.99
Dirty Ace	2006	147	29.99
Clyde Theory	2006	139	29.99
Clockwork Paradise	2006	143	29.99
Ballroom Mockingbird	2006	173	29.99

(25 rows)

Une instruction **SQL SELECT** et son résultat

Avant la généralisation du stockage et de la récupération numériques des données, **les fiches bristol** étaient utilisées pour **la conservation des données** dans de nombreux contextes : à la maison pour noter et conserver recettes, listes de courses, coordonnées et autres informations pratiques ; en entreprise pour consigner des notes de présentation, des recherches et des notes de projet, ainsi que des informations de contact ; à l'école comme aide-mémoire ou autres supports visuels ; et dans la recherche universitaire pour conserver des données telles que des références bibliographiques ou des notes dans un **fichier de fiches** . Les indexeurs professionnels utilisaient des fiches bristol pour la création d'index de livres jusqu'à leur remplacement par **des logiciels d'indexation** dans les années 1980 et 1990.

Les petites bases de données peuvent être stockées sur un **système de fichiers** , tandis que les grandes bases de données sont hébergées sur **des clusters de calcul** ou **dans le cloud** . La **conception des bases de données** englobe des techniques formelles et des considérations pratiques, notamment **la modélisation des données** , leur représentation et leur stockage

efficaces. Les langages de requêtes, la sécurité et la confidentialité des données sensibles, ainsi que les problématiques liées au calcul distribué, telles que la prise en charge des accès concurrents et la tolérance aux pannes.

Les informaticiens classent les systèmes de gestion de bases de données selon les modèles de bases de données qu'ils prennent en charge. Les bases de données relationnelles se sont imposées dans les années 1980. Elles modélisent les données sous forme de lignes et de colonnes dans une série de tables, et la grande majorité utilise SQL pour l'écriture et l'interrogation des données. Dans les années 2000, les bases de données non relationnelles, regroupées sous l'appellation NoSQL, ont gagné en popularité car elles utilisent des langages de requête différents.

Terminologie et aperçu

Formellement, une « base de données » désigne un ensemble de données liées entre elles, accessibles via un « système de gestion de base de données » (SGBD). Ce dernier est un ensemble intégré de logiciels permettant aux utilisateurs d'interagir avec une ou plusieurs bases de données et d'accéder à l'ensemble des données qu'elles contiennent (sous réserve de restrictions d'accès à certaines données). Le SGBD offre diverses fonctions permettant la saisie, le stockage et la récupération de grandes quantités d'informations, ainsi que des moyens de gérer l'organisation de ces informations.

Du fait de leur étroite relation, le terme « base de données » est souvent utilisé de manière informelle pour désigner à la fois une base de données et le SGBD utilisé pour la manipuler.

Outside the world of professional information technology, the term *database* is often used to refer to any collection of related data (such as a spreadsheet or a card index) as size and usage requirements typically necessitate use of a database management system.^[1]

Existing DBMSs provide various functions that allow management of a database and its data which can be classified into four main functional groups:

- **Data definition** – Creation, modification and removal of definitions that detail how the data is to be organized.
- **Update** – Insertion, modification, and deletion of the data itself.^[2]
- **Retrieval** – Selecting data according to specified criteria (e.g., a query, a position in a hierarchy, or a position in relation to other data) and providing that data either directly to the user, or making it available for further processing by the database itself or by other applications. The retrieved data may be made available in a more or less direct form without modification, as it is stored in the database, or in a new form obtained by altering it or combining it with existing data from the database.^[3]

Registering and monitoring users, enforcing data security, monitoring

performance, maintaining data integrity, dealing with concurrency control, and recovering

information that has been corrupted by some event such as an unexpected system failure.^[4]

Both a database and its DBMS conform to the principles of a particular [database model](#).^[5]

"Database system" refers collectively to the database model, database management system, and database.^[6]

Physically, database [servers](#) are dedicated computers that hold the actual databases and run only the DBMS and related software. Database servers are usually [multiprocessor](#) computers, with generous memory and [RAID](#) disk arrays used for stable storage. Hardware database accelerators, connected to one or more servers via a high-speed channel, are also used in large-volume [transaction processing environments](#). DBMSs are found at the heart of most [database applications](#). DBMSs may be built around a custom [multitasking kernel](#) with built-in [networking](#) support, but modern DBMSs typically rely on a standard [operating system](#) to provide these functions.

Since DBMSs comprise a significant [market](#), computer and storage vendors often take into account DBMS requirements in their own development plans.^[7]

Databases and DBMSs can be categorized according to the database model(s) that they support (such as [relational](#) or [XML](#)), the type(s) of computer they run on (from a [server cluster](#) to a [mobile phone](#)), the [query language](#)(s) used to access the database (such as SQL or [XQuery](#)), and their internal engineering, which affects performance, [scalability](#), resilience, and security.

Histoire

The sizes, capabilities, and performance of databases and their respective DBMSs have grown in orders of magnitude. These performance increases were enabled by the technology progress in the areas of [processors](#), [computer memory](#), [computer storage](#), and [computer networks](#). The concept of a database was made possible by the emergence of direct access [storage media](#) such as [magnetic disks](#), which became widely available in the mid-1960s; earlier systems relied on sequential storage of data on [magnetic tape](#). The subsequent development of database technology can be divided into three eras based on data model or structure: [navigational](#),^[8] SQL/[relational](#), and post-relational.

The two main early navigational [data models](#) were the [hierarchical model](#) and the [CODASYL model](#) ([network model](#)). These were characterized by the use of [pointers](#) (often physical disk addresses) to follow relationships from one record to another.

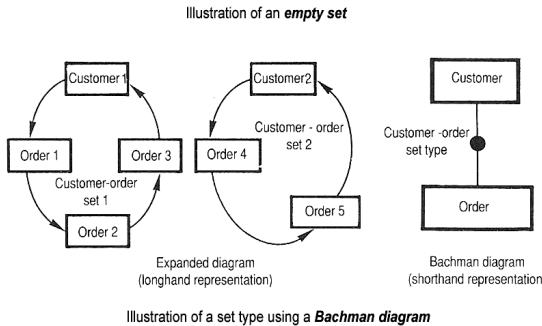
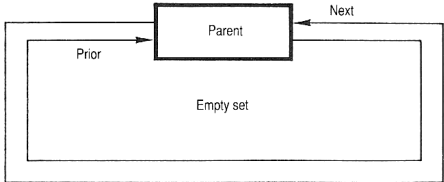
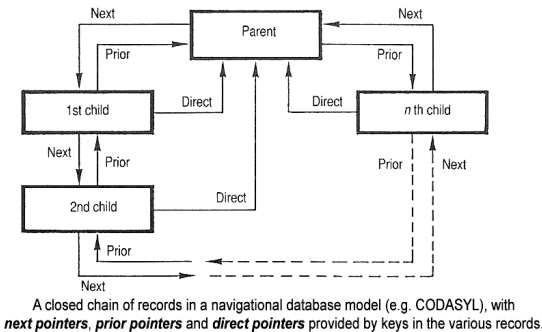
The [relational model](#), first proposed in 1970 by [Edgar F. Codd](#), departed from this tradition by insisting that [applications](#) should search for data by content, rather than by following links. The

relational model employs sets of ledger-style tables, each used for a different type of **entity**. Only in the mid-1980s did computing hardware become powerful enough to allow the wide deployment of relational systems (DBMSs plus applications). By the early 1990s, however, relational systems dominated in all large-scale **data processing** applications, and as of 2018 they remain dominant: **IBM Db2**, **Oracle**, **MySQL**, and **Microsoft SQL Server** are the most searched **DBMS**.^[9] The dominant database language, standardized SQL for the relational model, has influenced database languages for other data models.

Object databases were developed in the 1980s to overcome the inconvenience of **object–relational impedance mismatch**, which led to the coining of the term "post-relational" and also the development of hybrid **object–relational databases**.

The next generation of post-relational databases in the late 2000s became known as **NoSQL** databases, introducing fast **key–value stores** and **document-oriented databases**. A competing "next generation" known as **NewSQL** databases attempted new implementations that retained the relational/SQL model while aiming to match the high performance of NoSQL compared to commercially available relational DBMSs.

1960s, navigational DBMS



The record set, basic structure of navigational (e.g. CODASYL) database model. A set consists of one parent record (also called "the owner"), and n child records (also called members records)

Basic structure of navigational **CODASYL** database model

The introduction of the term *database* coincided with the availability of direct-access storage (disks and drums) from the mid-1960s onwards. The term represented a contrast with the tape-based systems of the past, allowing shared interactive use rather than daily [batch processing](#). The [Oxford English Dictionary](#) cites a 1962 report by the [System Development Corporation](#) of California as the first to use the term "data-base" in a specific technical sense.^[10]

As computers grew in speed and capability, a number of general-purpose database systems emerged; by the mid-1960s a number of such systems had come into commercial use. Interest in a standard began to grow, and [Charles Bachman](#), author of one such product, the [Integrated Data Store](#) (IDS), founded the [Database Task Group](#) within [CODASYL](#), the group responsible for the creation and standardization of [COBOL](#). In 1971, the Database Task Group delivered their standard, which generally became known as the *CODASYL approach*, and soon a number of commercial products based on this approach entered the market.

The CODASYL approach offered applications the ability to navigate around a linked data set which was formed into a large network. Applications could find records by one of three methods:

1. Use of a primary key (known as a CALC key, typically implemented by [hashing](#))
2. Navigating relationships (called *sets*) from one record to another
3. Scanning all the records in a sequential order

Later systems added [B-trees](#) to provide alternate access paths. Many CODASYL databases also added a declarative query language for end users (as distinct from the navigational [API](#)). However, CODASYL databases were complex and required significant training and effort to produce useful applications.

[IBM](#) also had its own DBMS in 1966, known as [Information Management System](#) (IMS). IMS was a development of software written for the [Apollo program](#) on the [System/360](#). IMS was generally similar in concept to CODASYL, but used a strict hierarchy for its model of data navigation instead of CODASYL's network model. Both concepts later became known as navigational databases due to the way data was accessed: the term was popularized by Bachman's 1973 [Turing Award](#) presentation *The Programmer as Navigator*. IMS is classified by IBM as a [hierarchical database](#). IDMS and [Cincom Systems'](#) [TOTAL](#) databases are classified as network databases. IMS remains in use as of 2014.^[11]

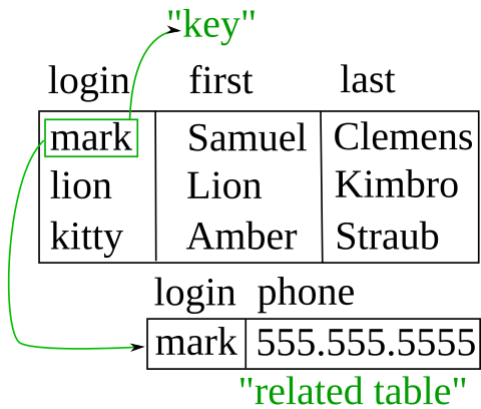
1970s, relational DBMS

[Edgar F. Codd](#) worked at IBM in [San Jose, California](#), in an office primarily involved in the development of [hard disk](#) systems.^[12] He was unhappy with the navigational model of the CODASYL approach, notably the lack of a "search" facility. In 1970, he wrote a number of papers

that outlined a new approach to database construction that eventually culminated in the groundbreaking *A Relational Model of Data for Large Shared Data Banks*.^[13]

The paper described a new system for storing and working with large databases. Instead of records being stored in some sort of **linked list** of free-form records as in CODASYL, Codd's idea was to organize the data as a number of "**tables**", each table being used for a different type of entity. Each table would contain a fixed number of columns containing the attributes of the entity. One or more columns of each table were designated as a **primary key** by which the rows of the table could be uniquely identified; cross-references between tables always used these primary keys, rather than disk addresses, and queries would join tables based on these key relationships, using a set of operations based on the mathematical system of **relational calculus** (from which the model takes its name). Splitting the data into a set of normalized tables (or *relations*) aimed to ensure that each "fact" was only stored once, thus simplifying update operations. Virtual tables called *views* could present the data in different ways for different users, but views could not be directly updated.

Codd used mathematical terms to define the model: relations, tuples, and domains rather than tables, rows, and columns. The terminology that is now familiar came from early implementations. Codd would later criticize the tendency for practical implementations to depart from the mathematical foundations on which the model was based.



In the **relational model**, records are "linked" using virtual keys not stored in the database but defined as needed between the data contained in the records.

The use of primary keys (user-oriented identifiers) to represent cross-table relationships, rather than disk addresses, had two primary motivations. From an engineering perspective, it enabled tables to be relocated and resized without expensive database reorganization. But Codd was more interested in the difference in semantics: the use of explicit identifiers made it easier to define update operations with clean mathematical definitions, and it also enabled query operations to be defined in terms of the established discipline of **first-order predicate calculus**; because these operations have clean mathematical properties, it becomes possible to rewrite queries in provably correct ways, which is the basis of query optimization. There is no loss of

expressions compared with the hierarchic or network models, though the connections between tables are no longer so explicit.

In the hierarchic and network models, records were allowed to have a complex internal structure. For example, the salary history of an employee might be represented as a "repeating group" within the employee record. In the relational model, the process of normalization led to such internal structures being replaced by data held in multiple tables, connected only by logical keys.

For instance, a common use of a database system is to track information about users, their name, login information, various addresses and phone numbers. In the navigational approach, all of this data would be placed in a single variable-length record. In the relational approach, the data would be *normalized* into a user table, an address table and a phone number table (for instance). Records would be created in these optional tables only if the address or phone numbers were actually provided.

As well as identifying rows/records using logical identifiers rather than disk addresses, Codd changed the way in which applications assembled data from multiple records. Rather than requiring applications to gather data one record at a time by navigating the links, they would use a declarative query language that expressed what data was required, rather than the access path by which it should be found. Finding an efficient access path to the data became the responsibility of the database management system, rather than the application programmer. This process, called query optimization, depended on the fact that queries were expressed in terms of mathematical logic.

Codd's paper inspired teams at various universities to research the subject, including one at [University of California, Berkeley](#)^[12] led by [Eugene Wong](#) and [Michael Stonebraker](#), who started [INGRES](#) using funding that had already been allocated for a geographical database project and student programmers to produce code. Beginning in 1973, INGRES delivered its first test products which were generally ready for widespread use in 1979. INGRES was similar to [System R](#) in a number of ways, including the use of a "language" for [data access](#), known as [QUEL](#). Over time, INGRES moved to the emerging SQL standard.

IBM itself did one test implementation of the relational model, [PRTV](#), and a production one, [Business System 12](#), both now discontinued. [Honeywell](#) wrote [MRDS](#) for [Multics](#), and now there are two new implementations: [Alphora Dataphor](#) and Rel. Most other DBMS implementations usually called *relational* are actually SQL DBMSs.

In 1970, the University of Michigan began development of the [MICRO Information Management System](#)^[14] based on [D.L. Childs'](#) Set-Theoretic Data model.^{[15][16][17]} The university in 1974 hosted a debate between Codd and Bachman which Bruce Lindsay of IBM later described as "throwing lightning bolts at each other!".^[12] MICRO was used to manage very large data sets by the [US Department of Labor](#), the [U.S. Environmental Protection Agency](#), and researchers from the

University of Alberta, the University of Michigan, and Wayne State University. It ran on IBM mainframe computers using the Michigan Terminal System.^[18] The system remained in production until 1998.

Integrated approach

In the 1970s and 1980s, attempts were made to build database systems with integrated hardware and software. The underlying philosophy was that such integration would provide higher performance at a lower cost. Examples were IBM System/38, the early offering of Teradata, and the Britton Lee, Inc. database machine.

Another approach to hardware support for database management was ICL's CAFS accelerator, a hardware disk controller with programmable search capabilities. In the long term, these efforts were generally unsuccessful because specialized database machines could not keep pace with the rapid development and progress of general-purpose computers. Thus most database systems nowadays are software systems running on general-purpose hardware, using general-purpose computer data storage. However, this idea is still pursued in certain applications by some companies like Netezza and Oracle (Exadata).

Late 1970s, SQL DBMS

IBM formed a team led by Codd that started working on a prototype system, System R despite opposition from others at the company.^[12] The first version was ready in 1974/5, and work then started on multi-table systems in which the data could be split so that all of the data for a record (some of which is optional) did not have to be stored in a single large "chunk". Subsequent multi-user versions were tested by customers in 1978 and 1979, by which time a standardized query language – SQL^[19] – had been added. Codd's ideas were establishing themselves as both workable and superior to CODASYL, pushing IBM to develop a true production version of System R, known as SQL/DS, and, later, Database 2 (IBM Db2).

Larry Ellison's Oracle Database (or more simply, Oracle) started from a different chain, based on IBM's papers on System R. Though Oracle V1 implementations were completed in 1978, it was not until Oracle Version 2 when Ellison beat IBM to market in 1979.^[20]

Stonebraker went on to apply the lessons from INGRES to develop a new database, Postgres, which is now known as PostgreSQL. PostgreSQL is often used for global mission-critical applications (the .org and .info domain name registries use it as their primary data store, as do many large companies and financial institutions).

In Sweden, [Codd's paper](#) was also read and [Mimer SQL](#) was developed in the mid-1970s at [Uppsala University](#). In 1984, this project was consolidated into an independent enterprise.

Another data model, the [entity–relationship model](#), emerged in 1976 and gained popularity for [database design](#) as it emphasized a more familiar description than the earlier relational model. Later on, entity–relationship constructs were retrofitted as a [data modeling](#) construct for the relational model, and the difference between the two has become irrelevant.

1980s, on the desktop

Besides IBM and various software companies such as [Sybase](#) and [Informix Corporation](#), most large computer hardware vendors by the 1980s had their own database systems such as [DEC's VAX Rdb/VMS](#).^[21] The decade ushered in the age of [desktop computing](#). The new computers empowered their users with spreadsheets like [Lotus 1-2-3](#) and database software like [dBASE](#). The dBASE product was lightweight and easy for any computer user to understand out of the box. [C. Wayne Ratliff](#), the creator of dBASE, stated: "dBASE was different from programs like BASIC, C, FORTRAN, and COBOL in that a lot of the dirty work had already been done. The data manipulation is done by dBASE instead of by the user, so the user can concentrate on what he is doing, rather than having to mess with the dirty details of opening, reading, and closing files, and managing space allocation."^[22] dBASE was one of the top selling software titles in the 1980s and early 1990s.

1990s, object-oriented

By the start of the decade databases had become a billion-dollar industry in about ten years.^[21] The 1990s, along with a rise in [object-oriented programming](#), saw a growth in how data in various databases were handled. Programmers and designers began to treat the data in their databases as [objects](#). That is to say that if a person's data were in a database, that person's attributes, such as their address, phone number, and age, were now considered to belong to that person instead of being extraneous data. This allows for relations between data to be related to objects and their [attributes](#) and not to individual fields.^[23] The term "[object–relational impedance mismatch](#)" described the inconvenience of translating between programmed objects and database tables. [Object databases](#) and [object–relational databases](#) attempt to solve this problem by providing an object-oriented language (sometimes as extensions to SQL) that programmers can use as alternative to purely relational SQL. On the programming side, libraries known as [object–relational mappings](#) (ORMs) attempt to solve the same problem.

2000s, NoSQL and NewSQL

Database sales grew rapidly during the [dotcom bubble](#) and, after its end, the rise of [ecommerce](#). The popularity of [open source](#) databases such as [MySQL](#) has grown since 2000, to the extent that Ken Jacobs of Oracle said in 2005 that perhaps "these guys are doing to us what we did to IBM".^[21]

[XML databases](#) are a type of structured document-oriented database that allows querying based on [XML](#) document attributes. XML databases are mostly used in applications where the data is conveniently viewed as a collection of documents, with a structure that can vary from the very flexible to the highly rigid: examples include scientific articles, patents, tax filings, and personnel records.

[NoSQL](#) databases are often very fast,^{[24][25]} do not require fixed table schemas, avoid join operations by storing [denormalized](#) data, and are designed to [scale horizontally](#).

In recent years, there has been a strong demand for massively distributed databases with high partition tolerance, but according to the [CAP theorem](#), it is impossible for a [distributed system](#) to simultaneously provide [consistency](#), availability, and partition tolerance guarantees. A distributed system can satisfy any two of these guarantees at the same time, but not all three. For that reason, many NoSQL databases are using what is called [eventual consistency](#) to provide both availability and partition tolerance guarantees with a reduced level of data consistency.

[NewSQL](#) is a class of modern relational databases that aims to provide the same scalable performance of NoSQL systems for online transaction processing (read-write) workloads while still using SQL and maintaining the [ACID](#) guarantees of a traditional database system.

Cas d'utilisation

Databases are used to support internal operations of organizations and to underpin online interactions with customers and suppliers (see [Enterprise software](#)).

Databases are used to hold administrative information and more specialized data, such as engineering data or economic models. Examples include computerized [library](#) systems, [flight reservation systems](#), computerized [parts inventory systems](#), and many [content management systems](#) that store [websites](#) as collections of webpages in a database.

Termes de classification

One way to classify databases involves the type of their contents, for example: [bibliographic](#), document-text, statistical, or multimedia objects. Another way is by their application area, for

even by inserting music compositions, movies, banking, manufacturing, or insurance. A third way is by some technical aspect, such as the database structure or interface type. This section lists a few of the adjectives used to characterize different kinds of databases.

- An **in-memory database** is a database that primarily resides in **main memory**, but is typically backed-up by non-volatile computer data storage. Main memory databases are faster than disk databases, and so are often used where response time is critical, such as in telecommunications network equipment.
- An **active database** includes an event-driven architecture which can respond to conditions both inside and outside the database. Possible uses include security monitoring, alerting, statistics gathering and authorization. Many databases provide active database features in the form of **database triggers**.
- A **cloud database** relies on **cloud technology**. Both the database and most of its DBMS reside remotely, "in the cloud", while its applications are both developed by programmers and later maintained and used by end-users through a **web browser** and **Open APIs**.
- **Data warehouses** archive data from operational databases and often from external sources such as market research firms. The warehouse becomes the central source of data for use by managers and other end-users who may not have access to operational data. For example, sales data might be aggregated to weekly totals and converted from internal product codes to use **UPCs** so that they can be compared with **ACNielsen** data. Some basic and essential components of data warehousing include extracting, analyzing, and **mining** data, transforming, loading, and managing data so as to make them available for further use.
- A **deductive database** combines **logic programming** with a relational database.
- A **distributed database** is one in which both the data and the DBMS span multiple computers.
- A **document-oriented database** is designed for storing, retrieving, and managing document-oriented, or semi structured, information. Document-oriented databases are one of the main categories of NoSQL databases.
- An **embedded database** system is a DBMS which is tightly integrated with an application software that requires access to stored data in such a way that the DBMS is hidden from the application's end-users and requires little or no ongoing maintenance.^[26]
- End-user databases consist of data developed by individual end-users. Examples of these are collections of documents, spreadsheets, presentations, multimedia, and other files. Several products exist to support such databases.
- A **federated database system** comprises several distinct databases, each with its own DBMS. It is handled as a single database by a federated database management system (FDBMS), which transparently integrates multiple autonomous DBMSs, possibly of different types (in

which case it would also be a [heterogeneous database system](#)), and provides them with an integrated conceptual view.

- Sometimes the term *multi-database* is used as a synonym for federated database, though it may refer to a less integrated (e.g., without an FDBMS and a managed integrated schema) group of databases that cooperate in a single application. In this case, typically [middleware](#) is used for distribution, which typically includes an atomic commit protocol (ACP), e.g., the [two-phase commit protocol](#), to allow [distributed \(global\) transactions](#) across the participating databases.
- A [graph database](#) is a kind of NoSQL database that uses [graph structures](#) with nodes, edges, and properties to represent and store information. General graph databases that can store any graph are distinct from specialized graph databases such as [triplestores](#) and [network databases](#).
- An [array DBMS](#) is a kind of NoSQL DBMS that allows modeling, storage, and retrieval of (usually large) multi-dimensional [arrays](#) such as satellite images and climate simulation output.
- In a [hypertext](#) or [hypermedia](#) database, any word or a piece of text representing an object, e.g., another piece of text, an article, a picture, or a film, can be [hyperlinked](#) to that object. Hypertext databases are particularly useful for organizing large amounts of disparate information. For example, they are useful for organizing [online encyclopedias](#), where users can conveniently jump around the text. The [World Wide Web](#) is thus a large distributed hypertext database.
- A [knowledge base](#) (abbreviated **KB**, **kb** or Δ ^{[27][28]}) is a special kind of database for [knowledge management](#), providing the means for the computerized collection, organization, and [retrieval](#) of [knowledge](#). Also a collection of data representing problems with their solutions and related experiences.
- A [mobile database](#) can be carried on or synchronized from a mobile computing device.
- [Operational databases](#) store detailed data about the operations of an organization. They typically process relatively high volumes of updates using [transactions](#). Examples include [customer databases](#) that record contact, credit, and demographic information about a business's customers, personnel databases that hold information such as salary, benefits, skills data about employees, [enterprise resource planning](#) systems that record details about product components, parts inventory, and financial databases that keep track of the organization's money, accounting and financial dealings.
- A [parallel database](#) seeks to improve performance through [parallelization](#) for tasks such as loading data, building indexes and evaluating queries.

The major parallel DBMS architectures which are induced by the underlying [hardware](#) architecture are:

[Shared memory architecture](#), where multiple processors share the main memory space, as well as other data storage.

- **Shared disk architecture**, where each processing unit (typically consisting of multiple processors) has its own main memory, but all units share the other storage.
- **Shared-nothing architecture**, where each processing unit has its own main memory and other storage.
- [Probabilistic databases](#) employ [fuzzy logic](#) to draw inferences from imprecise data.
- [Real-time databases](#) process transactions fast enough for the result to come back and be acted on right away.
- A [spatial database](#) can store the data with multidimensional features. The queries on such data include location-based queries, like "Where is the closest hotel in my area?".
- A [temporal database](#) has built-in time aspects, for example a temporal data model and a temporal version of [SQL](#). More specifically the temporal aspects usually include valid-time and transaction-time.
- A [terminology-oriented database](#) builds upon an [object-oriented database](#), often customized for a specific field.
- An [unstructured data](#) database is intended to store in a manageable and protected way diverse objects that do not fit naturally and conveniently in common databases. It may include email messages, documents, journals, multimedia objects, etc. The name may be misleading since some objects can be highly structured. However, the entire possible object collection does not fit into a predefined structured framework. Most established DBMSs now support unstructured data in various ways, and new dedicated DBMSs are emerging.

système de gestion de base de données

Connolly and Begg define database management system (DBMS) as a "software system that enables users to define, create, maintain and control access to the database."^[29] Examples of DBMS's include [MySQL](#), [MariaDB](#), [PostgreSQL](#), [Microsoft SQL Server](#), [Oracle Database](#), and [Microsoft Access](#).

The DBMS acronym is sometimes extended to indicate the underlying [database model](#), with RDBMS for the [relational](#), OODBMS for the [object \(oriented\)](#) and ORDBMS for the [object–relational model](#). Other extensions can indicate some other characteristics, such as DDBMS for a distributed database management systems.

The functionality provided by a DBMS can vary enormously. The core functionality is the storage, retrieval and update of data. [Codd](#) proposed the following functions and services a fully-fledged

general purpose DBMS should provide:^[30]

- Data storage, retrieval and update
- User accessible catalog or [data dictionary](#) describing the metadata
- Support for transactions and concurrency
- Facilities for recovering the database should it become damaged
- Support for authorization of access and update of data
- Access support from remote locations
- Enforcing constraints to ensure data in the database abides by certain rules

It is also generally to be expected the DBMS will provide a set of utilities for such purposes as may be necessary to administer the database effectively, including import, export, monitoring, defragmentation and analysis utilities.^[31] The core part of the DBMS interacting between the database and the application interface sometimes referred to as the [database engine](#).

Often DBMSs will have configuration parameters that can be statically and dynamically tuned, for example the maximum amount of main memory on a server the database can use. The trend is to minimize the amount of manual configuration, and for cases such as [embedded databases](#) the need to target zero-administration is paramount.

The large major enterprise DBMSs have tended to increase in size and functionality and have involved up to thousands of human years of development effort throughout their lifetime.^[a]

Early multi-user DBMS typically only allowed for the application to reside on the same computer with access via [terminals](#) or terminal emulation software. The [client-server architecture](#) was a development where the application resided on a client desktop and the database on a server allowing the processing to be distributed. This evolved into a [multitier architecture](#) incorporating [application servers](#) and [web servers](#) with the end user interface via a [web browser](#) with the database only directly connected to the adjacent tier.^[33]

A general-purpose DBMS will provide public [application programming interfaces](#) (API) and optionally a processor for [database languages](#) such as [SQL](#) to allow applications to be written to interact with and manipulate the database. A special purpose DBMS may use a private API and be specifically customized and linked to a single application. For example, an [email](#) system performs many of the functions of a general-purpose DBMS such as message insertion, message deletion, attachment handling, blocklist lookup, associating messages an email address and so forth however these functions are limited to what is required to handle email.

Application

External interaction with the database will be via an application program that interfaces with the DBMS.^[34] This can range from a [database tool](#) that allows users to execute SQL queries textually or graphically, to a website that happens to use a database to store and search information.

Application program interface

A [programmer](#) will [code](#) interactions to the database (sometimes referred to as a [datasource](#)) via an [application program interface](#) (API) or via a [database language](#). The particular API or language chosen will need to be supported by DBMS, possibly indirectly via a [preprocessor](#) or a bridging API. Some API's aim to be database independent, [ODBC](#) being a commonly known example. Other common API's include [JDBC](#) and [ADO.NET](#).

Langages de bases de données

Database languages are special-purpose languages, which allow one or more of the following tasks, sometimes distinguished as [sublanguages](#):

- [Data control language](#) (DCL) – controls access to data;
- [Data definition language](#) (DDL) – defines data types such as creating, altering, or dropping tables and the relationships among them;
- [Data manipulation language](#) (DML) – performs tasks such as inserting, updating, or deleting data occurrences;
- [Data query language](#) (DQL) – allows searching for information and computing derived information.

Database languages are specific to a particular data model. Notable examples include:

- SQL combines the roles of data definition, data manipulation, and query in a single language. It was one of the first commercial languages for the relational model, although it departs in some respects from [the relational model as described by Codd](#) (for example, the rows and columns of a table can be ordered). SQL became a standard of the [American National Standards Institute](#) (ANSI) in 1986, and of the [International Organization for Standardization](#) (ISO) in 1987. The standards have been regularly enhanced since and are supported (with varying degrees of conformance) by all mainstream commercial relational DBMSs.^{[35][36]}
- [OQL](#) is an object model language standard (from the [Object Data Management Group](#)). It has influenced the design of some of the newer query languages like [JDOQL](#) and [EJB QL](#).

[MarkLogic](#) and [eXist](#), by relational databases with XML capability such as Oracle and Db2, and also by in-memory XML processors such as [Saxon](#).

- [SQL/XML](#) combines [XQuery](#) with SQL.^[37]

A database language may also incorporate features like:

- DBMS-specific configuration and storage engine management
- Computations to modify query results, like counting, summing, averaging, sorting, grouping, and cross-referencing
- Constraint enforcement (e.g. in an automotive database, only allowing one engine type per car)
- Application programming interface version of the query language, for programmer convenience

Stockage

Database storage is the container of the physical materialization of a database. It comprises the *internal* (physical) *level* in the database architecture. It also contains all the information needed (e.g., [metadata](#), "data about the data", and internal [data structures](#)) to reconstruct the *conceptual level* and *external level* from the internal level when needed. Databases as digital objects contain three layers of information which must be stored: the data, the structure, and the semantics. Proper storage of all three layers is needed for future [preservation](#) and longevity of the database.^[38] Putting data into permanent storage is generally the responsibility of the [database engine](#) a.k.a. "storage engine". Though typically accessed by a DBMS through the underlying operating system (and often using the operating systems' [file systems](#) as intermediates for storage layout), storage properties and configuration settings are extremely important for the efficient operation of the DBMS, and thus are closely maintained by database administrators. A DBMS, while in operation, always has its database residing in several types of storage (e.g., memory and external storage). The database data and the additional needed information, possibly in very large amounts, are coded into bits. Data typically reside in the storage in structures that look completely different from the way the data look at the conceptual and external levels, but in ways that attempt to optimize (the best possible) these levels' reconstruction when needed by users and programs, as well as for computing additional types of needed information from the data (e.g., when querying the database).

Some DBMSs support specifying which [character encoding](#) was used to store data, so multiple encodings can be used in the same database.

Various low-level database storage structures are used by the storage engine to serialize the data model so it can be written to the medium of choice. Techniques such as indexing may be

used to improve performance. Conventional storage is row-oriented, but there are also [column-oriented](#) and [correlation databases](#).

Materialized views

Often storage redundancy is employed to increase performance. A common example is storing *materialized views*, which consist of frequently needed *external views* or query results. Storing such views saves the expensive computing them each time they are needed. The downsides of materialized views are the overhead incurred when updating them to keep them synchronized with their original updated database data, and the cost of storage redundancy.

Replication

Occasionally a database employs storage redundancy by [database objects](#) replication (with one or more copies) to increase data availability (both to improve performance of simultaneous multiple end-user accesses to the same database object, and to provide resiliency in a case of partial failure of a distributed database). Updates of a replicated object need to be synchronized across the object copies. In many cases, the entire database is replicated.

Virtualization

With [data virtualization](#), the data used remains in its original locations and real-time access is established to allow analytics across multiple sources. This can aid in resolving some technical difficulties such as compatibility problems when combining data from various platforms, lowering the risk of error caused by faulty data, and guaranteeing that the newest data is used. Furthermore, avoiding the creation of a new database containing personal information can make it easier to comply with privacy regulations. However, with data virtualization, the connection to all necessary data sources must be operational as there is no local copy of the data, which is one of the main drawbacks of the approach. ^[39]

Sécurité

[Database security](#) deals with all various aspects of protecting the database content, its owners, and its users. It ranges from protection from intentional unauthorized database uses to unintentional database accesses by unauthorized entities (e.g., a person or a computer program).

Database access control deals with controlling who (a person or a certain computer program) are allowed to access what information in the database. The information may comprise specific database objects (e.g., record types, specific records, data structures), certain computations over certain objects (e.g., query types, or specific queries), or using specific access paths to the former (e.g., using specific indexes or other data structures to access information). Database access controls are set by special authorized (by the database owner) personnel that uses dedicated protected security DBMS interfaces.

This may be managed directly on an individual basis, or by the assignment of individuals and **privileges** to groups, or (in the most elaborate models) through the assignment of individuals and groups to roles which are then granted entitlements. Data security prevents unauthorized users from viewing or updating the database. Using passwords, users are allowed access to the entire database or subsets of it called "subschemas". For example, an employee database can contain all the data about an individual employee, but one group of users may be authorized to view only payroll data, while others are allowed access to only work history and medical data. If the DBMS provides a way to interactively enter and update the database, as well as interrogate it, this capability allows for managing personal databases.

Data security in general deals with protecting specific chunks of data, both physically (i.e., from corruption, or destruction, or removal; e.g., see **physical security**), or the interpretation of them, or parts of them to meaningful information (e.g., by looking at the strings of bits that they comprise, concluding specific valid credit-card numbers; e.g., see **data encryption**).

Change and access logging records who accessed which attributes, what was changed, and when it was changed. Logging services allow for a forensic **database audit** later by keeping a record of access occurrences and changes. Sometimes application-level code is used to record changes rather than leaving this in the database. Monitoring can be set up to attempt to detect security breaches. Therefore, organizations must take database security seriously because of the many benefits it provides. Organizations will be safeguarded from security breaches and hacking activities like firewall intrusion, virus spread, and ransom ware. This helps in protecting the company's essential information, which cannot be shared with outsiders at any cause.^[40]

Transactions et concurrence

Database transactions can be used to introduce some level of **fault tolerance** and **data integrity** after recovery from a **crash**. A database transaction is a unit of work, typically encapsulating a number of operations over a database (e.g., reading a database object, writing, acquiring or releasing a **lock**, etc.), an abstraction supported in database and also other systems. Each transaction has well defined boundaries in terms of which program/code executions are included

in that transaction (determined by the transaction's programmer via special transaction commands).

The acronym [ACID](#) describes some ideal properties of a database transaction: [atomicity](#), [consistency](#), [isolation](#), and [durability](#).

Migration

A database built with one DBMS is not portable to another DBMS (i.e., the other DBMS cannot run it). However, in some situations, it is desirable to migrate a database from one DBMS to another. The reasons are primarily economical (different DBMSs may have different [total costs of ownership](#) or TCOs), functional, and operational (different DBMSs may have different capabilities). The migration involves the database's transformation from one DBMS type to another. The transformation should maintain (if possible) the database related application (i.e., all related application programs) intact. Thus, the database's conceptual and external architectural levels should be maintained in the transformation. It may be desired that also some aspects of the architecture internal level are maintained. A complex or large database migration may be a complicated and costly (one-time) project by itself, which should be factored into the decision to migrate. This is in spite of the fact that tools may exist to help migration between specific DBMSs. Typically, a DBMS vendor provides tools to help import databases from other popular DBMSs.

Construction, entretien et réglage

After designing a database for an application, the next stage is building the database. Typically, an appropriate general-purpose DBMS can be selected to be used for this purpose. A DBMS provides the needed [user interfaces](#) to be used by database administrators to define the needed application's data structures within the DBMS's respective data model. Other user interfaces are used to select needed DBMS parameters (like security related, storage allocation parameters, etc.).

When the database is ready (all its data structures and other needed components are defined), it is typically populated with initial application's data (database initialization, which is typically a distinct project; in many cases using specialized DBMS interfaces that support bulk insertion) before making it operational. In some cases, the database becomes operational while empty of application data, and data are accumulated during its operation.

After the database is created, initialized and populated it needs to be maintained. Various database parameters may need changing and the database may need to be tuned ([tuning](#)) for

better performance; application's data structures may be changed or added, new related application programs may be written to add to the application's functionality, etc.

Sauvegarde et restauration

Sometimes it is desired to bring a database back to a previous state (for many reasons, e.g., cases when the database is found corrupted due to a software error, or if it has been updated with erroneous data). To achieve this, a backup operation is done occasionally or continuously, where each desired database state (i.e., the values of its data and their embedding in database's data structures) is kept within dedicated backup files (many techniques exist to do this effectively). When it is decided by a database administrator to bring the database back to this state (e.g., by specifying this state by a desired point in time when the database was in this state), these files are used to restore that state.

Analyse statique

Static analysis techniques for software verification can be applied also in the scenario of query languages. In particular, the *[Abstract interpretation](#) framework has been extended to the field of query languages for relational databases as a way to support sound approximation techniques.^[41] The semantics of query languages can be tuned according to suitable abstractions of the concrete domain of data. The abstraction of relational database systems has many interesting applications, in particular, for security purposes, such as fine-grained access control, watermarking, etc.

Caractéristiques diverses

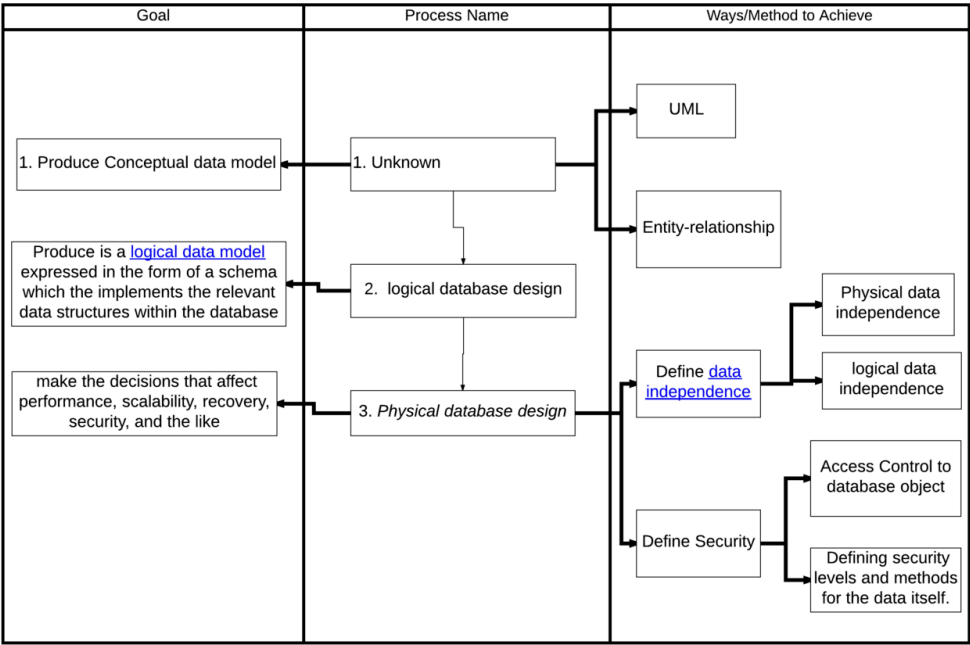
Other DBMS features might include:

- [Database logs](#) – This helps in keeping a history of the executed functions.
- Graphics component for producing graphs and charts, especially in a data warehouse system.
- [Query optimizer](#) – Performs query optimization on every query to choose an efficient [query plan](#) (a partial order (tree) of operations) to be executed to compute the query result. May be specific to a particular storage engine.
- Tools or hooks for database design, application programming, application program maintenance, database performance analysis and monitoring, database configuration monitoring, DBMS hardware configuration (a DBMS and related database may span computers, networks, and storage units) and related database mapping (especially for a distributed DBMS), storage allocation and database layout monitoring, storage migration, etc.

NOM : RASOLOFOMALALA
PRENOM : Maminiaina Angelo
NUMERO D'INSCRIPTION : 29625

Increasingly, there are calls for a single system that incorporates all of these core functionalities into the same build, test, and deployment framework for database management and source control. Borrowing from other developments in the software industry, some market such offerings as "DevOps for database".^[42]

Conception et modélisation



The first task of a database designer is to produce a [conceptual data model](#) that reflects the structure of the information to be held in the database. A common approach to this is to develop an [entity-relationship model](#), often with the aid of drawing tools. Another popular approach is the [Unified Modeling Language](#). A successful data model will accurately reflect the possible state of the external world being modeled: for example, if people can have more than one phone number, it will allow this information to be captured. Designing a good conceptual data model requires a good understanding of the application domain; it typically involves asking deep questions about the things of interest to an organization, like "can a customer also be a supplier?", or "if a product is sold with two different forms of packaging, are those the same product or different products?", or "if a plane flies from New York to Dubai via Frankfurt, is that one flight or two (or maybe even three)?". The answers to these questions establish definitions of the terminology used for entities (customers, products, flights, flight segments) and their relationships and attributes.

Producing the conceptual data model sometimes involves input from [business processes](#), or the analysis of [workflow](#) in the organization. This can help to establish what information is needed in the database, and what can be left out. For example, it can help when deciding whether the database needs to hold historic data as well as current data.

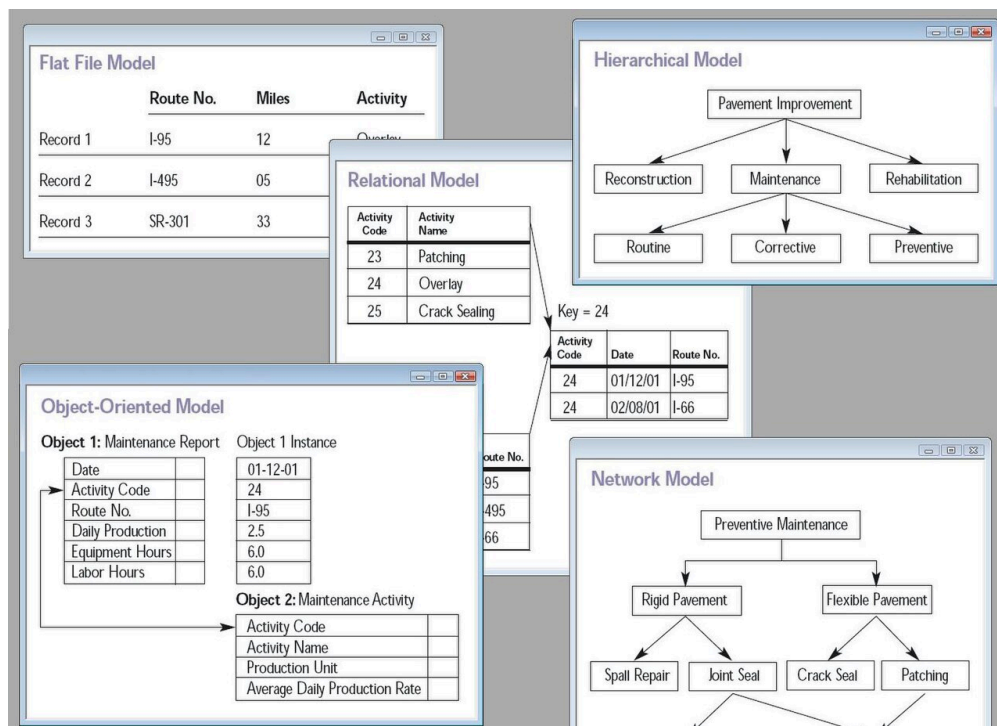
Having produced a conceptual data model that users are happy with, the next stage is to translate this into a [schema](#) that implements the relevant data structures within the database. This process is often called logical database design, and the output is a [logical data model](#) expressed in the form of a schema. Whereas the conceptual data model is (in theory at least) independent of the choice of database technology, the logical data model will be expressed in terms of a particular database model supported by the chosen DBMS. (The terms *data model* and *database model* are often used interchangeably, but in this article we use *data model* for the design of a specific database, and *database model* for the modeling notation used to express that design).

The most popular database model for general-purpose databases is the relational model, or more precisely, the relational model as represented by the SQL language. The process of creating a logical database design using this model uses a methodical approach known as [normalization](#). The goal of normalization is to ensure that each elementary "fact" is only recorded in one place, so that insertions, updates, and deletions automatically maintain consistency.

The final stage of database design is to make the decisions that affect performance, scalability, recovery, security, and the like, which depend on the particular DBMS. This is often called *physical database design*, and the output is the [physical data model](#). A key goal during this stage is [data independence](#), meaning that the decisions made for performance optimization purposes should be invisible to end-users and applications. There are two types of data independence: Physical data independence and logical data independence. Physical design is driven mainly by performance requirements, and requires a good knowledge of the expected workload and access patterns, and a deep understanding of the features offered by the chosen DBMS.

Another aspect of physical database design is security. It involves both defining [access control](#) to database objects as well as defining security levels and methods for the data itself.

Models



Collage of five types of database models

A database model is a type of data model that determines the logical structure of a database and fundamentally determines in which manner **data** can be stored, organized, and manipulated. The most popular example of a database model is the relational model (or the SQL approximation of relational), which uses a table-based format.

Common logical data models for databases include:

- **Navigational databases**
 - Hierarchical database model
 - Network model
 - Graph database
- Relational model
- Entity-relationship model
 - Enhanced entity-relationship model
- Object model
- Document model
- Entity-attribute-value model
- Star schema

An object-relational database combines the two related structures.

- [Inverted index](#)
- [Flat file](#)

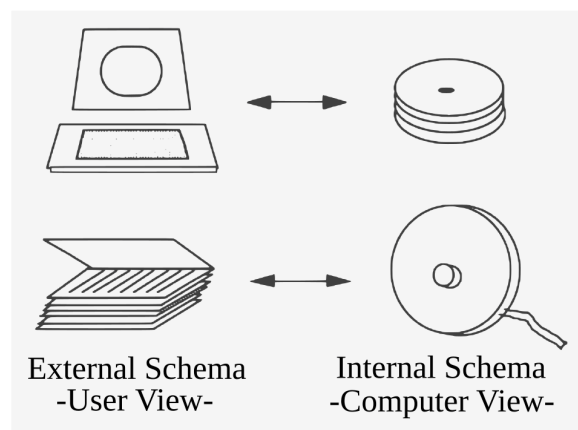
Other models include:

- [Multidimensional model](#)
- [Array model](#)
- [Multivalue model](#)

Specialized models are optimized for particular types of data:

- [XML database](#)
- [Semantic model](#)
- [Content store](#)
- [Event store](#)
- [Time series model](#)

External, conceptual, and internal views



Traditional view of data^[43]

A database management system provides three views of the database data:

- The **external level** defines how each group of end-users sees the organization of data in the database. A single database can have any number of views at the external level.
- The **conceptual level** (or *logical level*) unifies the various external views into a compatible global view.^[44] It provides the synthesis of all the external views. It is out of the scope of the various database end-users, and is rather of interest to database application developers and database administrators.

The **internal level** (or *physical level*) is the internal organization of data inside a DBMS. It is concerned with cost, performance, scalability and other operational matters. It deals with storage layout of the data, using storage structures such as **indexes** to enhance performance. Occasionally it stores data of individual views (**materialized views**), computed from generic data, if performance justification exists for such redundancy. It balances all the external views' performance requirements, possibly conflicting, in an attempt to optimize overall performance across all activities.

While there is typically only one conceptual and internal view of the data, there can be any number of different external views. This allows users to see database information in a more business-related way rather than from a technical, processing viewpoint. For example, a financial department of a company needs the payment details of all employees as part of the company's expenses, but does not need details about employees that are in the interest of the **human resources** department. Thus different departments need different *views* of the company's database.

The three-level database architecture relates to the concept of *data independence* which was one of the major initial driving forces of the relational model.^[44] The idea is that changes made at a certain level do not affect the view at a higher level. For example, changes in the internal level do not affect application programs written using conceptual level interfaces, which reduces the impact of making physical changes to improve performance.

The conceptual view provides a level of indirection between internal and external. On the one hand it provides a common view of the database, independent of different external view structures, and on the other hand it abstracts away details of how the data are stored or managed (internal level). In principle every level, and even every external view, can be presented by a different data model. In practice usually a given DBMS uses the same data model for both the external and the conceptual levels (e.g., relational model). The internal level, which is hidden inside the DBMS and depends on its implementation, requires a different level of detail and uses its own types of data structure types.

Recherche

Database technology has been an active research topic since the 1960s, both in academia and in the research and development groups of companies (for example **IBM Research**). Research activity includes **theory** and development of **prototypes**. Notable research topics have included **models**, the atomic transaction concept, related **concurrency control** techniques, query languages and **query optimization** methods, **RAID**, and more.

The database research area has several dedicated **academic journals** (for example, **ACM Transactions on Database Systems**-TODS, **Data and Knowledge Engineering**-DKE) and annual

Voir aussi

- [Comparison of database tools](#)
- [Comparison of object database management systems](#)
- [Comparison of object–relational database management systems](#)
- [Comparison of relational database management systems](#)
- [Data bank](#) – Organized collection of data in computing
- [Data hierarchy](#) – Systematic organization of data
- [Data store](#) – Repository for data collection storage and management
- [Database testing](#) – Testing of database software systems
- [Database theory](#) – Study of database design and use
- [Database-as-IPC](#) – Misusing databases for temporary messages
- [Database-centric architecture](#) – Software architecture
- [Datalog](#) – Declarative logic programming language
- [DBOS](#) – Commercial services on top of an open source library that provides Durable Computing
- [Flat-file database](#) – Database stored as flat data
- [INP \(database\)](#) – Early database management system
- [Journal of Database Management](#)
- [Casio Databank](#) – Brand of watch
- [Conformational dynamics data bank](#) – Database of protein conformations
- [Data repository](#) – Long-term storage of research data
- [Databank Systems](#) – Financial computing shared service
- [Dortmund Data Bank](#)
- [Electron Microscopy Data Bank](#) – European biological database
- [Hazardous Substances Data Bank](#) – Database of toxic compounds
- [List of databases](#)
- [Memory bank](#) – Logical unit of storage in computer architecture

NOM : RASOLOFOMALALA

PRENOM : Maminiaina Angelo

NUMERO D'INSCRIPTION : 296125

[National Trauma Data Bank](#) – Compilation of U.S. traumatic injury data from participating institutions

- [Protein Data Bank](#) – International open access database of large biological molecules
- [Star Wars Databank](#)

Notes

- a. This article quotes a development time of five years involving 750 people for DB2 release 9 alone.^[32]

Références

1. [Ullman & Widom 1997](#), p. 1.
2. ["Update Definition & Meaning"](#) (<http://www.merriam-webster.com/dictionary/update>) . *Merriam-Webster*. Archived (<https://web.archive.org/web/20240225065959/https://www.merriam-webster.com/dictionary/update>) from the original on Feb 25, 2024.
3. ["Retrieval Definition & Meaning"](#) (<http://www.merriam-webster.com/dictionary/retrieval>) . *Merriam-Webster*. Archived (<https://web.archive.org/web/20230627174611/https://www.merriam-webster.com/dictionary/retrieval>) from the original on Jun 27, 2023.
4. ["Administration Definition & Meaning"](#) (<http://www.merriam-webster.com/dictionary/administration>) . *Merriam-Webster*. Archived (<https://web.archive.org/web/20231206055116/http://www.merriam-webster.com/dictionary/administration>) from the original on Dec 6, 2023.
5. [Tsitchizris & Lochovsky 1982](#).
6. [Beynon-Davies 2003](#).
7. [Nelson & Nelson 2001](#).
8. [Bachman 1973](#).
9. ["TOPDB Top Database index"](#) (<https://pypl.github.io/DB.html>) . *pypl.github.io*.
10. ["database, n"](#) (<http://www.oed.com/view/Entry/47411>) . *OED Online*. Oxford University Press. June 2013. Retrieved July 12, 2013. (Subscription required.)
11. IBM Corporation (October 2013). ["IBM Information Management System \(IMS\) 13 Transaction and Database Servers delivers high performance and low total cost of ownership"](#) (<http://www-01.ibm.com/common/ssi/cgi-bin/ssialias?subtype=ca&infotype=a&n&appname=iSource&supplier=897&letternum=ENUS213-381>) . Retrieved Feb 20, 2014.

12. "RDBMS Plenary 1: Early Years" (<https://archive.computerhistory.org/resources/access/text/2013/05/102702562-05-01-acc.pdf>) (PDF) (Interview). Interviewed by Burton Grad. Computer History Museum. 2007-06-12. Retrieved 2025-05-30.
13. Codd 1970.
14. Hershey & Easthope 1972.
15. North 2010.
16. Childs 1968a.
17. Childs 1968b.
18. M.A. Kahn; D.L. Rumelhart; B.L. Bronson (October 1977). *MICRO Information Management System (Version 5.0) Reference Manual* (https://docs.google.com/viewer?a=v&pid=explorer&chrome=true&srcid=0B4t_NX-QeWDYZGMwOTRmOTItZTg2Zi00YmJkLTg4MTktN2E4MWU0YmZlMjE3) . Institute of Labor and Industrial Relations (ILIR), University of Michigan and Wayne State University.
19. "What Is Structured Query Language (SQL)?" (<https://www.ibm.com/think/topics/structured-query-language>) . IBM. Retrieved 2026-03-18. "In the 1970s, IBM scientists Donald Chamberlin and Raymond Boyce developed and introduced SQL. It originated from the concept of relational models and was initially called structured English query language (SEQUEL) before being shortened to SQL."
20. "Oracle 30th Anniversary Timeline" (<https://www.oracle.com/us/corporate/profit/p27anniv-timeline-151918.pdf>) (PDF). Archived (<https://web.archive.org/web/20110320220813/http://www.oracle.com/us/corporate/profit/p27anniv-timeline-151918.pdf>) (PDF) from the original on 2011-03-20. Retrieved 23 August 2017.
21. "RDBMS Plenary Session: The Later Years" (<https://archive.computerhistory.org/resources/access/text/2013/05/102701921-05-01-acc.pdf>) (PDF) (Interview). Interviewed by Burton Grad. Computer History Museum. 2007-06-12. Retrieved 2025-05-30.
22. Interview with Wayne Ratliff (http://www.foxprohistory.org/interview_wayne_ratliff.htm) . The FoxPro History. Retrieved on 2013-07-12.
23. Development of an object-oriented DBMS; Portland, Oregon, United States; Pages: 472–482; 1986; ISBN 0-89791-204-7
24. Jordan, Meghan. "NoSQL Latency" (<https://www.scylladb.com/glossary/nosql-latency/>) . ScyllaDB. Retrieved 2025-06-09.
25. "SQL vs. NoSQL: Full comparison of features, differences, and more" (<https://www.testgorilla.com/blog/sql-vs-nosql/>) . www.testgorilla.com. Retrieved 2025-06-09.

26. Steven Stone, "COTS Databases For Embedded Systems" (<http://www.embedded-computing.com/articles/id/?2020>) Archived (<https://web.archive.org/web/20071114050734/http://www.embedded-computing.com/articles/id/?2020>) 2007-11-14 at the [Wayback Machine](#), *Embedded Computing Design* magazine, January 2007. Retrieved on August 13, 2008.
27. Argumentation in Artificial Intelligence by Iyad Rahwan, Guillermo R. Simari
28. "OWL DL Semantics" (<http://www.obitko.com/tutorials/ontologies-semantic-web/owl-dl-semantics.html>) . Retrieved 10 December 2010.
29. Connolly & Begg 2014, p. 64.
30. Connolly & Begg 2014, pp. 97–102.
31. Connolly & Begg 2014, p. 102.
32. Chong et al. 2007.
33. Connolly & Begg 2014, pp. 106–113.
34. Connolly & Begg 2014, p. 65.
35. Chapple 2005.
36. "Structured Query Language (SQL)" (<http://publib.boulder.ibm.com/infocenter/db2luw/v9/index.jsp?topic=com.ibm.db2.udb.admin.doc/doc/c0004100.htm>) . International Business Machines. October 27, 2006. Retrieved 2007-06-10.
37. Wagner 2010.
38. Ramalho, J.C.; Faria, L.; Helder, S.; Coutada, M. (31 December 2013). "Database Preservation Toolkit: A flexible tool to normalize and give access to databases" (https://core.ac.uk/display/55635702?algorithmId=15&similarToDoc=55614406&similarToDocKey=CORE&recSetID=f3ffea4d-1504-45e9-bfd6-a0495f5c8f9c&position=2&recommendation_type=same_repo&otherRecs=55614407,55635702,55607961,55613627,2255664) . *Biblioteca Nacional de Portugal*. University of Minho.
39. Paiho, Satu; Tuominen, Pekka; Rökman, Jyri; Ylikerälä, Markus; Pajula, Juha; Siikavirta, Hanne (2022). "Opportunities of collected city data for smart cities" (<https://doi.org/10.1049%2Fsmc2.12044>) . *IET Smart Cities*. **4** (4): 275–291. doi:10.1049/smc2.12044 (<https://doi.org/10.1049%2Fsmc2.12044>) . ISSN 2631-7680 (<https://search.worldcat.org/issn/2631-7680>) . S2CID 253467923 (<https://api.semanticscholar.org/CorpusID:253467923>) .
40. David Y. Chan; Victoria Chiu; Miklos A. Vasarhelyi (2018). *Continuous auditing : theory and application* (1st ed.). Bingley, UK: Emerald Publishing. ISBN 978-1-78743-413-4. OCLC 1029759767 (<https://search.worldcat.org/oclc/1029759767>) .
41. Halder & Cortesi 2011.

42. Beynon-Davies (January 28, 2016). "How Database Administration Fits into DevOps" (<https://www.infoq.com/news/2016/01/database-administration-devops>) . Retrieved April 15, 2017.
43. itl.nist.gov (1993) *Integration Definition for Information Modeling (IDEFIX)* (<http://www.itl.nist.gov/fipspubs/idef1x.doc>) Archived (<https://web.archive.org/web/20131203223034/http://www.itl.nist.gov/fipspubs/idef1x.doc>) 2013-12-03 at the Wayback Machine. 21 December 1993.
44. Date 2003, pp. 31–32.

Sources

- Bachman, Charles W. (1973). "The Programmer as Navigator" (<https://doi.org/10.1145%2F355611.362534>) . *Communications of the ACM*. **16** (11): 653–658. doi:10.1145/355611.362534 (<https://doi.org/10.1145%2F355611.362534>) .
- Beynon-Davies, Paul (2003). *Database Systems* (3rd ed.). Palgrave Macmillan. ISBN 978-1403916013.
- Chapple, Mike (2005). "SQL Fundamentals" (<http://databases.about.com/od/sql/a/sqlfundamentals.htm>) . *Databases*. About.com. Archived (<https://web.archive.org/web/20090222225300/http://databases.about.com/od/sql/a/sqlfundamentals.htm>) from the original on 22 February 2009. Retrieved 28 January 2009.
- Childs, David L. (1968a). *Description of a set-theoretic data structure* (<https://deepblue.lib.umich.edu/bitstream/handle/2027.42/4163/bac0294.0001.001.pdf?sequence=5&isAllowed=y>) (PDF) (Technical report). CONCOMP (Research in Conversational Use of Computers) Project. University of Michigan. Technical Report 3.
- Childs, David L. (1968b). *Feasibility of a set-theoretic data structure: a general structure based on a reconstituted definition* (<https://deepblue.lib.umich.edu/bitstream/handle/2027.42/4164/bac0293.0001.001.pdf?sequence=5&isAllowed=y>) (PDF) (Technical report). CONCOMP (Research in Conversational Use of Computers) Project. University of Michigan. Technical Report 6.
- Chong, Raul F.; Wang, Xiaomei; Dang, Michael; Snow, Dwaine R. (2007). "Introduction to DB2" (<http://www.ibmpressbooks.com/articles/article.asp?p=1163083>) . *Understanding DB2: Learning Visually with Examples* (2nd ed.). IBM Press Pearson plc. ISBN 978-0131580183. Retrieved 17 March 2013.
- Codd, Edgar F. (1970). "A Relational Model of Data for Large Shared Data Banks" (<http://www.seas.upenn.edu/~zives/03f/cis550/codd.pdf>) (PDF). *Communications of the ACM*. **13** (6): 377–387. doi:10.1145/362384.362685 (<https://doi.org/10.1145%2F362384.362685>) . S2CID 207549016 (<https://api.semanticscholar.org/CorpusID:207549016>) .

Copyright: Thomas M. Begg, Carolyn E. (2014). *Database Systems – A Practical Approach to Design Implementation and Management* (6th ed.). Pearson. ISBN 978-1292061184.

- Date, C. J. (2003). *An Introduction to Database Systems* (<https://archive.org/details/introductionda0000date>) (8th ed.). Pearson. ISBN 978-0321197849.
- Halder, Raju; Cortesi, Agostino (2011). "Abstract Interpretation of Database Query Languages" (<https://web.archive.org/web/20241123005344/https://www.dsi.unive.it/~cortesi/paperi/CL2012.pdf>) (PDF). *Computer Languages, Systems & Structures*. **38** (2): 123–157. doi:10.1016/j.cl.2011.10.004 (<https://doi.org/10.1016%2Fj.cl.2011.10.004>) . ISSN 1477-8424 (<https://search.worldcat.org/issn/1477-8424>) . Archived from the original (<http://www.dsi.unive.it/~cortesi/paperi/CL2012.pdf>) (PDF) on 2024-11-23. Retrieved 2015-06-18.
- Hershey, William; Easthope, Carol (1972). *A set theoretic data structure and retrieval language* (https://docs.google.com/open?id=0B4t_NX-QeWDYNmVhYjAwMWMtYzc3ZS00YjI0LWJhMjgtZTYyODZmNmFkNThh) . Spring Joint Computer Conference, May 1972. *ACM SIGIR Forum*. Vol. 7, no. 4. pp. 45–55. doi:10.1145/1095495.1095500 (<https://doi.org/10.1145%2F1095495.1095500>) .
- Nelson, Anne Fulcher; Nelson, William Harris Morehead (2001). *Building Electronic Commerce: With Web Database Constructions*. Prentice Hall. ISBN 978-0201741308.
- North, Ken (10 March 2010). "Sets, Data Models and Data Independence" (<http://drdobbs.com/blogs/database/228700616>) . *Dr. Dobb's*. Archived (<https://web.archive.org/web/20121024064523/http://www.drdobbs.com/database/sets-data-models-and-data-independence/228700616>) from the original on 24 October 2012.
- Tsitchizris, Dionysios C.; Lochovsky, Fred H. (1982). *Data Models* (<https://archive.org/details/damamodels00tsic>) . Prentice–Hall. ISBN 978-0131964280.
- Ullman, Jeffrey; Widom, Jennifer (1997). *A First Course in Database Systems* (<https://archive.org/details/firstcourseindat00ullm>) . Prentice–Hall. ISBN 978-0138613372.
- Wagner, Michael (2010), *SQL/XML:2006 – Evaluierung der Standardkonformität ausgewählter Datenbanksysteme*, Diplomica Verlag, ISBN 978-3836696098

Pour en savoir plus

- Ling Liu and Tamer M. Özsu (Eds.) (2009). "Encyclopedia of Database Systems" (<https://www.springer.com/computer/database+management+&+information+retrieval/book/978-0-387-49616-0>) , 4100 p. 60 illus. ISBN 978-0-387-49616-0.
- Gray, J. and Reuter, A. *Transaction Processing: Concepts and Techniques*, 1st edition, Morgan Kaufmann Publishers, 1992.

NOM : RASOLOFOMALALA

PRENOM : Maminiaina Angelo

NUMERO D'INSCRIPTION : 29625

M. Kroenke, David M. and David J. Auer. *Database Concepts*. 3rd ed. New York: Prentice, 2007.

- Raghu Ramakrishnan and Johannes Gehrke, *Database Management Systems* (<http://pages.cs.wisc.edu/~dbbook/>) .
- Abraham Silberschatz, Henry F. Korth, S. Sudarshan, *Database System Concepts* (<http://www.db-book.com/>) .
- Lightstone, S.; Teorey, T.; Nadeau, T. (2007). *Physical Database Design: the database professional's guide to exploiting indexes, views, storage, and more*. Morgan Kaufmann Press. ISBN 978-0-12-369389-1.
- Teorey, T.; Lightstone, S. and Nadeau, T. *Database Modeling & Design: Logical Design*, 4th edition, Morgan Kaufmann Press, 2005. ISBN 0-12-685352-5.
- *CMU Database courses playlist* (<https://www.youtube.com/@CMUDatabaseGroup/playlists>)
- *MIT OCW 6.830 | Fall 2010 | Database Systems* (<https://ocw.mit.edu/courses/6-830-database-systems-fall-2010/>)
- *Berkeley CS W186* (<https://cs186berkeley.net>)

Liens externes

- [DB File extension \(http://www.fileextension.org/DB\)](http://www.fileextension.org/DB) – information about files with the DB extension